

注意力机制那么多，到底该选哪个？

剑桥、帝国理工与牛津提出 DARTFormer：用可微架构搜索为长序列任务挑注意力，并回答了一个更尖锐的问题——混合多种注意力会更好吗？

自 Transformer 问世以来，研究者为它的自注意力(self-attention)设计了一长串更省算力的替代品：Longformer、Bigbird、Performer、Reformer、Linformer、Synthesizer.....原始点积注意力的计算量随序列长度平方增长，这些高效变体正是为了把长序列任务的开销降下来。

问题是：面对一个具体的长序列任务，到底该用哪一种？此前的 Long Range Arena(LRA)基准已经给出一个让人不安的结论——在没有预训练、从零开始训练的设定下，最好的注意力高度依赖任务，没有哪一种能通吃。于是实践者只剩下一条笨办法：把每种注意力都训一遍再挑，代价高得离谱。

这篇 DARTFormer 想把“挑注意力”这件事做得又快又靠谱，同时顺手回答一个长期悬而未决的直觉问题：既然不同的注意力头(attention head)可能各自擅长不同的关系模式，那把多种注意力混在一起，会不会比任何单一注意力都强？论文的做法是把可微架构搜索(DARTS 式的 NAS)搬进 Transformer，先用一个“超网络”把所有候选注意力并联起来，再用一个便宜的打分指标把它们排序。

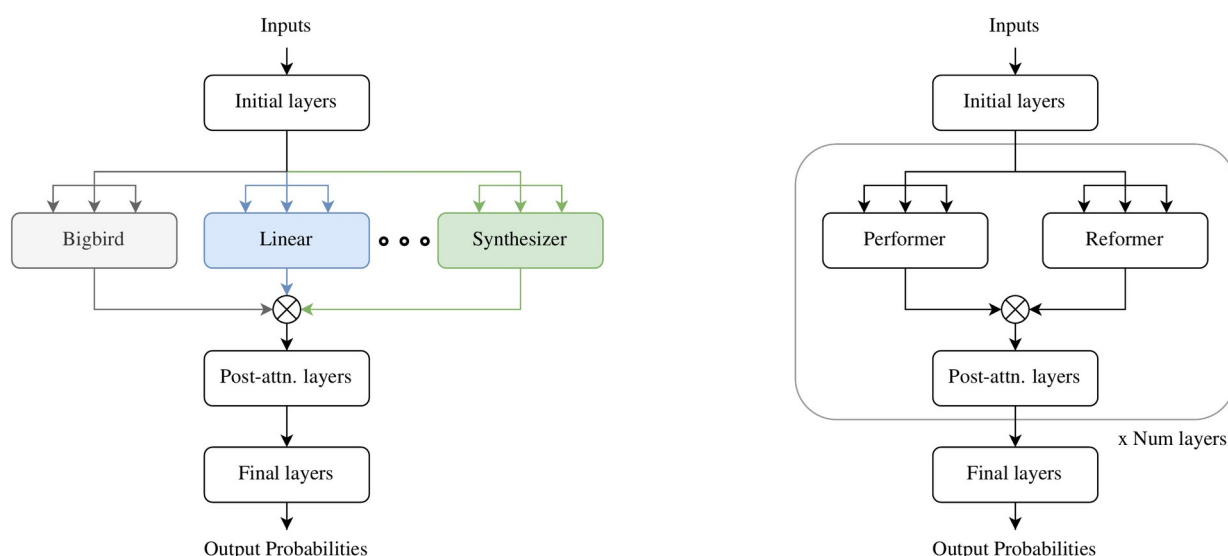


图 1 DARTFormer 的超网络(左)与搜索得到的最终架构(右)。超网络是一个单层 Transformer，把 Bigbird、Linear、Synthesizer 等多种高效注意力块并联，再对它们的输出取平均；右侧是搜索后导出的模型，在同一层里混合了 Performer 和 Reformer 两种注意力——跨注意力头异构、跨层同构。

论文: <https://arxiv.org/abs/2210.00641>

为什么"挑注意力"值得单独研究

在图像领域，卷积核会自动学到不同的特征；类似地，人们长期相信 Transformer 的不同注意力头会学到不同的关系偏置(relational bias)。如果这个直觉成立，那么一个同时容纳多种注意力类型的模型，理应能学到更丰富的关系，从而超过任何单一注意力的模型。DARTFormer 没有把这个直觉当成前提，而是把它当成一个需要被验证的假设，直接去问：一个任务的最优注意力，真的是多种注意力的混合吗？

要回答这个问题，暴力枚举显然行不通——九种注意力、还要考虑它们的各种组合，逐一训练的成本无法承受。这正是把架构搜索引进来的动机：用一次训练同时评估所有候选，而不是训练所有候选。

核心方法：并联所有注意力，再看少了谁最疼

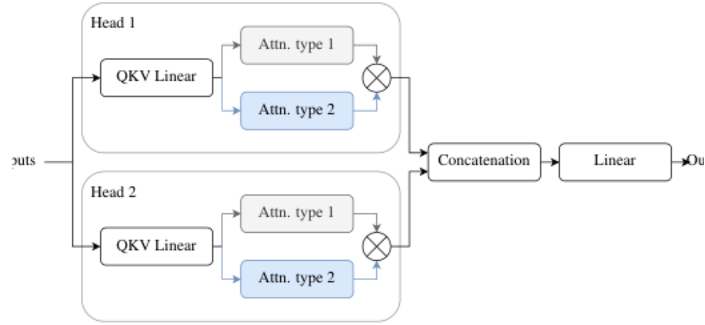
DARTFormer 的第一步是搭一个单层"超网络"：把每一种高效注意力都做成一个候选块(block)，让它们并排接收同样的输入。这里借用了 DARTS 的一个简化技巧——固定 α (fixed- α)：不去学习 softmax 边权重、也不做代价高昂的双层优化(bi-level optimization)，而是直接把所有候选块的输出取平均。公式上就是把 N 个候选块的输出简单平均， $\bar{o} = (1/N) \cdot \sum o_i$ 。好处是这个超网络可以一路训练到收敛，稳定又便宜。

训练好之后，怎么给每种注意力打分？论文用的指标叫"掩码验证精度下降"(masked validation accuracy drop)：把某一个注意力块屏蔽掉，看验证集精度掉多少——掉得越多，说明这个块越重要。删掉谁最疼，谁就是这个任务的最优注意力。对于只想要单一注意力的场景(同构模型)，取得分最高的那一个，放进完整的 Transformer 就行。

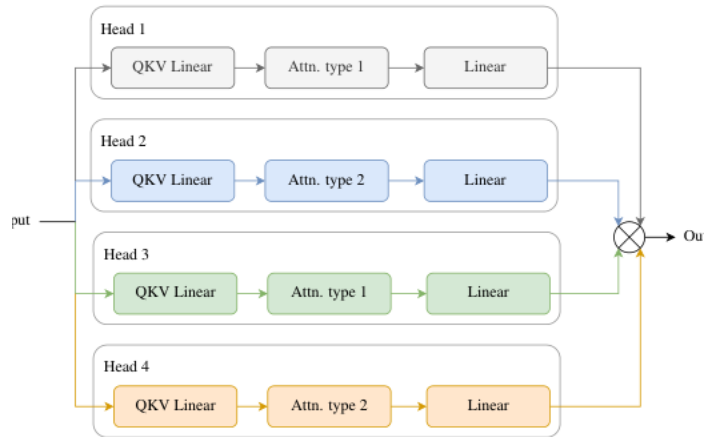
这个便宜的打分指标出人意料地灵。以 LRA 的 ListOps 任务为例，Reformer 的掩码验证下降得分高达 11.85，而其余八种注意力全部低于 0.5——这是一个干净利落、几乎没有歧义的选择信号。在文本分类任务上，搜索同样正确地把 Performer 选了出来。当然它并非万能：在得分整体偏小的文档匹配(document matching)任务上，单层超网络就没能把真正的前三名稳定选进自己的前三。

更进一步：能不能搜出一个"混合注意力"的模型

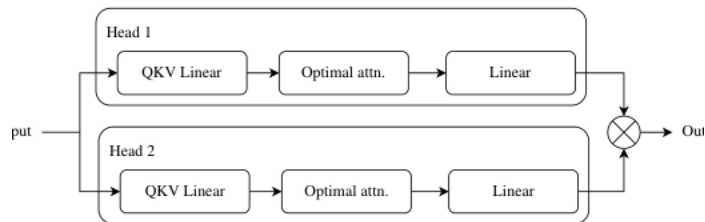
既然超网络天生就把多种注意力放在一起，很自然会想：干脆保留多种、拼成一个异构(heterogeneous)Transformer，会不会更强？论文把搜索空间下沉到注意力头这一层——让每个头固定使用一种注意力，整层再对各头的输出取平均。图 4 对比了这个想法的"理论版"和"实际实现版"，以及搜索结束后每个头都换成最优注意力的最终形态。



(a) Theoretical multi head attention supernet



(b) Actual multi head attention supernet



(c) Final multi attention layer after architecture search

图 4 注意力头层面的搜索空间。理论版(a)让每个头内部再去挑注意力类型;实际实现(b)改为每个头固定一种注意力、整层对各头输出取平均;(c)是搜索完成后每个头都替换为“最优注意力”的最终多头注意力层。

怎么从这个空间里挑出好的混合?论文给了两条路。一条叫 NAS Prune：从一个每个头都装着不同注意力的多头层出发，反复删掉得分最低的那个头、再微调，直到只剩下表现最好的若干头(图 3)。这条路准确但昂贵。另一条叫 NAS One-shot：干脆不迭代，直接从单层选择里取得分最高的四种注意力拼起来——便宜得多。

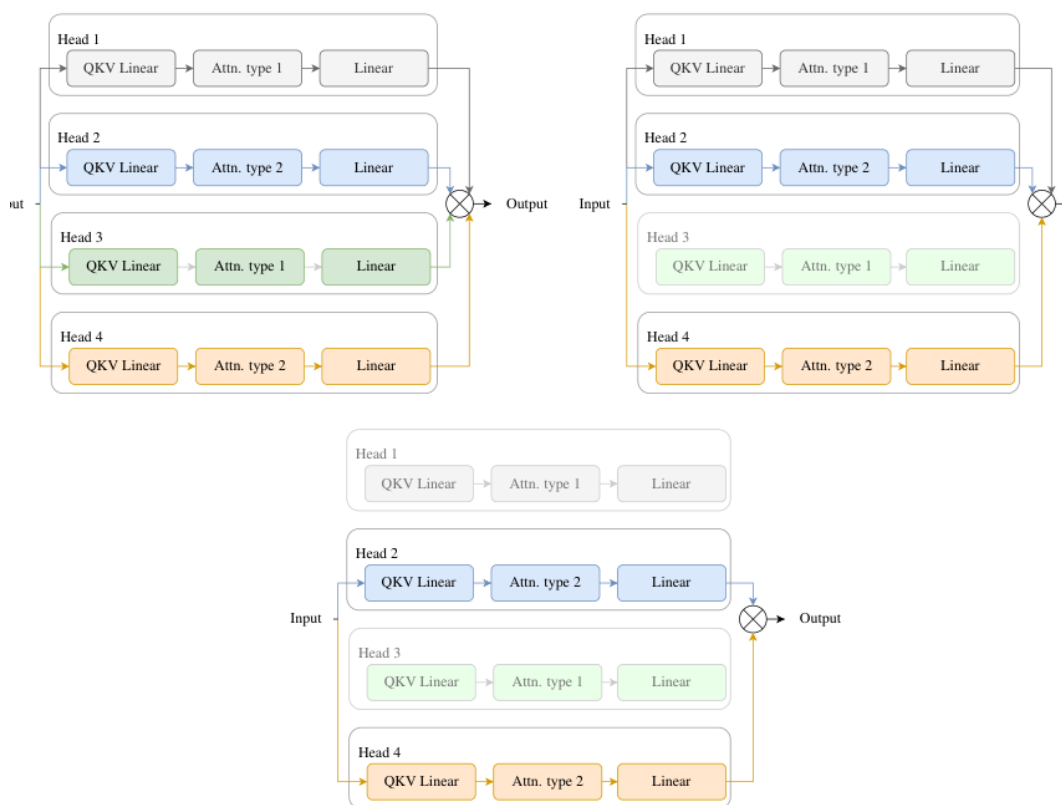


图 3 NAS Prune 的迭代剪枝过程：从一个四头、每头装有不同注意力的混合层出发，反复删掉得分最低的注意力头并微调，直到只留下最强的几个头。

关键结果：混合能赢过"平均"，但赢不过"最好"

结论分成方向相反的两半。好消息是，掩码验证下降这个搜索指标在挑"单一最优注意力"上是真的好用——文本分类和 ListOps 上它都选对了。坏消息是，当作者把多种注意力拼成异构 Transformer 后，无论用昂贵的 NAS Prune 还是便宜的 NAS One-shot，在全部三个任务上都没能超过最好的那个同构模型。

表 1 异构混合注意力模型 vs. 各任务上最优的同构模型(完整模型的测试精度 %)。三个任务上，**NAS Prune** 和 **NAS One-shot** 都没有超过"最优同构"这一列。

任务	最优同构模型	NAS Prune(异构)	NAS One-shot(异构)
文本分类	Performer 64.5	63.9	64.4
ListOps	Reformer 37.8	36.3	37.1
文档匹配	Synthesizer 71.1	67.0	64.7

数据来自论文 Table 2。文档匹配任务上差距最大，异构模型落后约 4–6 个百分点。

一个有用的实践旁证来自两种搜索方法的对比：昂贵的 NAS Prune(反复剪枝加微调)相对便宜的 NAS One-shot(单遍取前四)并没有稳定的优势——只要好的注意力被正确识别出来，便宜的方法就够用了。作者进一步尝试把混合向最强的注意力倾斜、或给强注意力分配更多头，同样没能带来一致的提升。

这告诉我们什么

对一个新的长序列任务，与其把九种注意力全训一遍，DARTFormer 给了一条便宜可靠的配方：训练一个单层、多注意力并联的 Transformer，用掩码验证精度下降来挑赢家。一个提醒是：如果所有候选的下降得分都很小(比如文档匹配那种量级)，这个选择信号就可能不可靠，需要谨慎。

更值得回味的是那个反直觉的结论。混合多种注意力，确实能打败“随便挑一个”的平均水平，却始终打不过“挑得最好”的那一个。这与“不同注意力偏置会互补叠加”的常见假设相冲突——至少在这些长序列 NLP 任务上，与其把多种注意力揉在一起，不如踏踏实实为任务选对那一种。当然，论文的边界也很清楚：结论建立在 LRA 的三个任务、九种候选注意力、从零训练(无预训练)的设置之上，换到别的任务或引入预训练后是否依旧成立，仍是开放问题。